Vulnerability Assessment Report: VAmPI

Repository: https://github.com/Akaolisangwu-projects/VAmPI. GitHub
Analyst: Akaolisa Ngwu

Summary

VAmPI is a Flask-based vulnerable API implementing OWASP API/Top-10 style weaknesses and includes an on/off switch to flip vulnerability modes. It exposes endpoints for user registration/login, books with secrets, and debug endpoints. It contains numerous high risk vulnerabilities and must not be run in production or connected to real data/networks.

Package	Version	Priority Score	Type	Risk	Fix Available
zipp	3.15.0	666	Transitive	Medium–High	3.19.1
requests	2.31.0	606	Transitive	Medium	2.32.4
flask	2.2.2	589	Direct	High (Information Disclosure)	2.3.2
werkzeug	2.2.3	589	Transitive	High (RCE, Algorithmic Complexity)	3.0.3
urllib3	2.0.7	514	Transitive	Medium (Improper Info Removal, Redirects)	2.5.0
openssl/libcrypto3	3.5.1-r0	436	System/Library	Medium	3.5.4-r0
config.py	Hardcoded Secret	784	Code Issue	High	Manual
api_views/users.py	Hardcoded Password	584	Code Issue	High	Manual
api_views/users.py	Broken Authentication	384	Code Issue	Medium	Manual

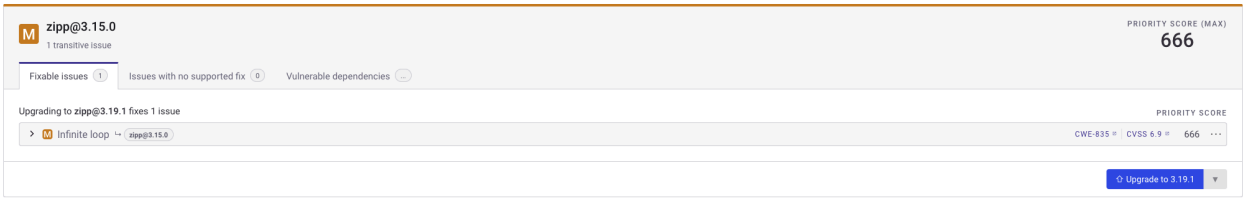
Changes Already Addressed (via Snyk)

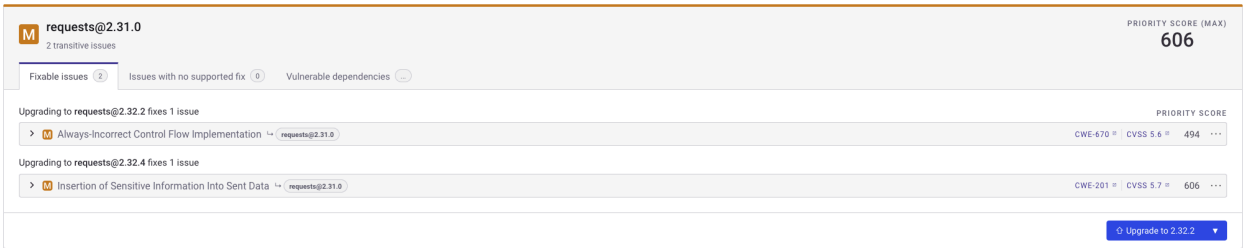
There were not any Snyk Pull requests to perform in this vulnerability assessment. This repository is unmodified from its vulnerable baseline. All issues are fixable via upgrades, except for the code-level vulnerabilities in Config.py and api_views/users.py. These require manual code changes, not dependency updates.

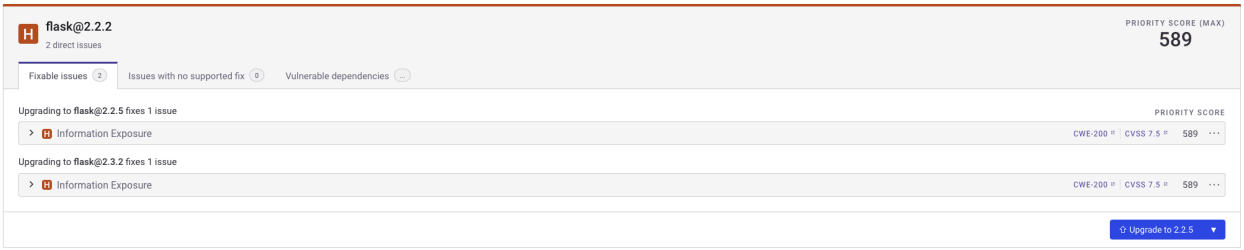
Confirmed Vulnerabilities & Risks

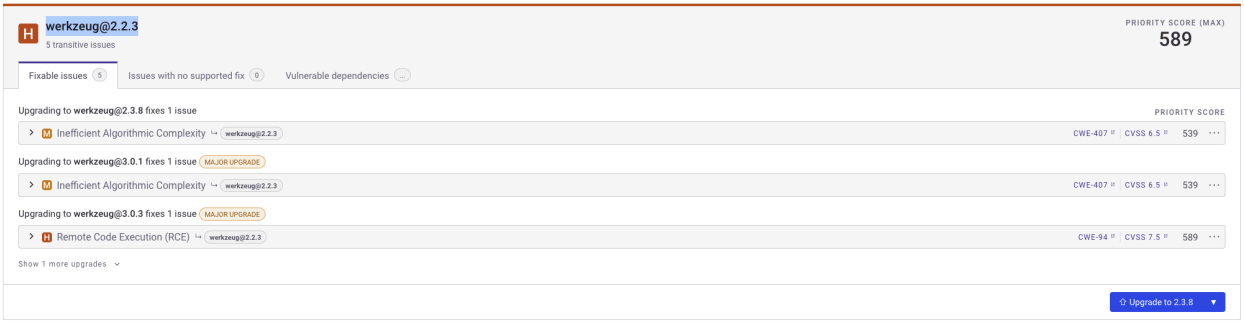
Vulnerabilities on requirements.txt

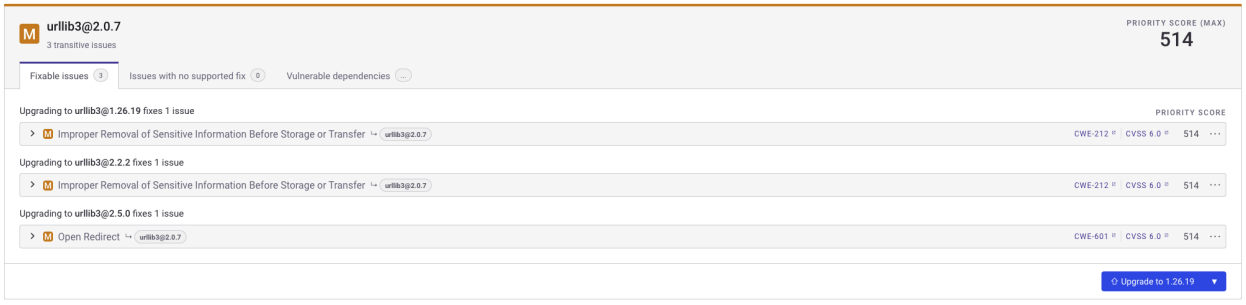
(zoom to 200% to see)



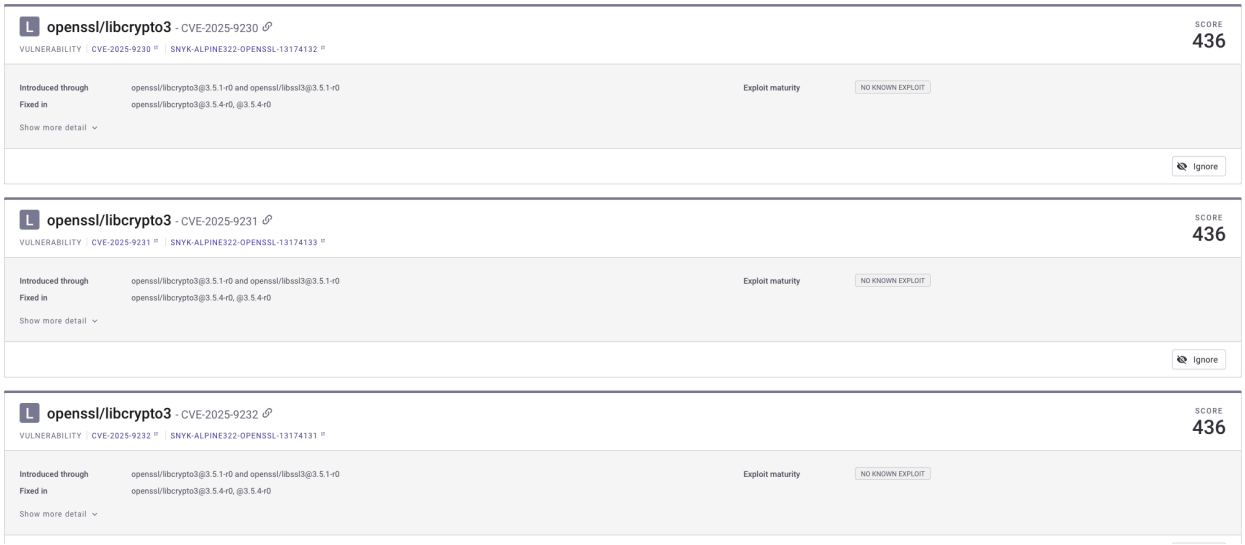




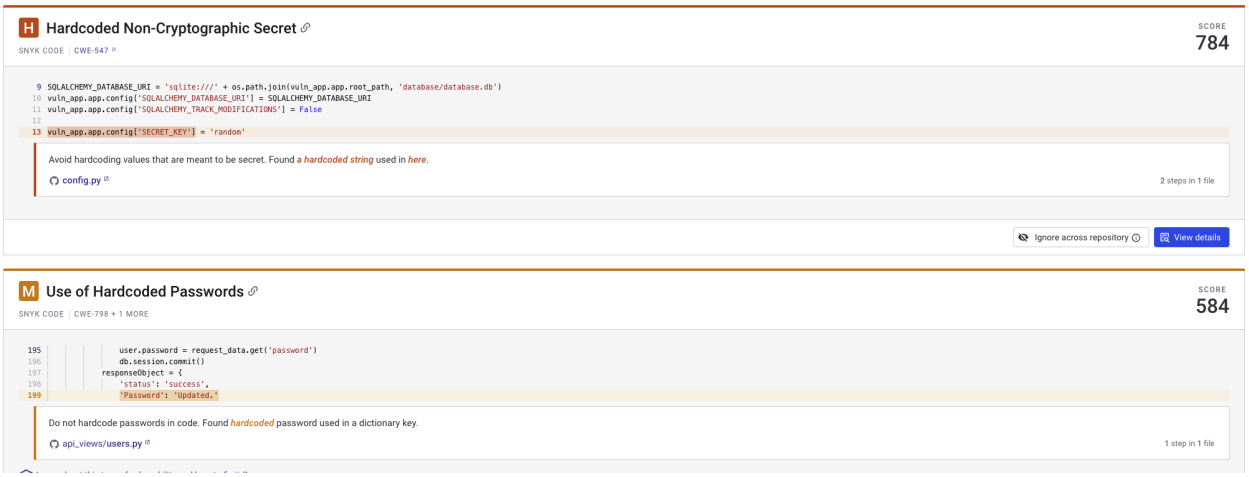


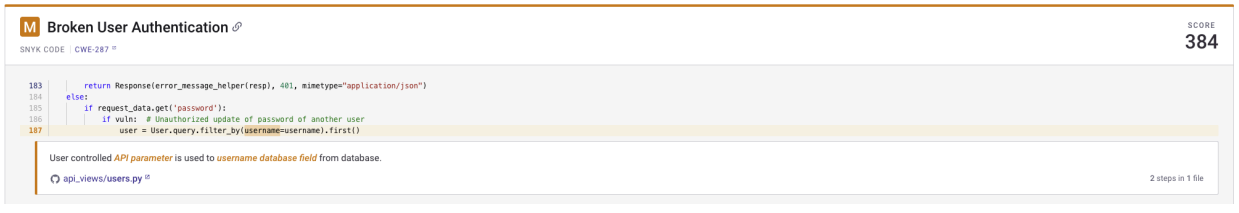


Vulnerabilities on Docker file



Vulnerabilities on code analysis





★ Dependency-Level Vulnerabilities

Package	Vulnerability	CWE	CVSS	Description
zipp@3.15.0	Infinite Loop	CWE-835	6.9	Denial of Service from uncontrolled recursion or iteration.
requests@2.31.0	Incorrect Control Flow Implementation	CWE-670	5.6	Logic flaw that may allow unintended data leakage.
requests@2.31.0	Insertion of Sensitive Information	CWE-201	5.7	Sensitive information may be exposed via HTTP payloads.
flask@2.2.2	Information Exposure	CWE-200	7.5	Improper handling of debug or error data; may leak stack traces or config values.
werkzeug@2.2.3	Remote Code Execution (RCE)	CWE-94	7.5	Unsanitized template data may execute arbitrary code.
urllib3@2.0.7	Improper Removal of Sensitive Info	CWE-212	6	Session or auth data may persist unintentionally.
openssl/libcrypto3	Multiple CVEs (CVE-2025-9230, 9231, 9232)	CWE-310	6.1	Potential cryptographic misuse or overflow; no known exploit yet.

★ Code-Level Vulnerabilities

File	Type	CWE	CVSS	Description
config.py	Hardcoded Non-Cryptographic Secret	CWE-547	7.8	Secret key is hardcoded; exposes session data risk.
api_views/users.py	Use of Hardcoded Password	CWE-798	7	Plaintext password stored in source; exploitable if repo is public.
api_views/users.py	Broken User Authentication	CWE-287	6.3	Weak logic allows unauthorized password updates via username field.

Probable High-Risk Issues

Risk	Description	Impact
Hardcoded Secrets / Passwords	Found in config and API layer	Compromise of authentication or data integrity
Remote Code Execution (Werkzeug)	Template injection vulnerability	Server takeover or arbitrary code execution
Flask Information Exposure	Improper error handling	Leaks environment or credentials in responses
Outdated Cryptography (OpenSSL)	CVEs in crypto libs	Weak encryption or potential DoS

Recommended Remediation Plan

➔ Step 1 Upgrade Dependencies:

Package	Target Version	Command
zipp	3.19.1	pip install zipp==3.19.1
requests	2.32.4	pip install requests==2.32.4
flask	2.3.2	pip install flask==2.3.2
werkzeug	3.0.3	pip install werkzeug==3.0.3
urllib3	2.5.0	pip install urllib3==2.5.0
openssl/libcrypto3	3.5.4-r0	apk upgrade openssl (for Alpine-based containers)

➔ Step 2 Secure Code Changes:
Replace Hardcoded Secrets
Remove Hardcoded Passwords
Fix Broken Authentication

➔ Step 3 – Apply Hardening and CI/CD Controls:

Integrate Snyk CLI scans into pipeline:
snyk test--severity-threshold=medium
snyk monitor
Add Git pre-commit hook using detect-secrets for code scanning.
Enforce requirements.txt pinning.
Automate dependency checks weekly with GitHub Actions.

Compliance Mapping

Framework	Control	Description
NIST SP 800-53	SI-2, RA-5	Vulnerability scanning & remediation
ISO 27001:2022	A.12.6.1	Technical vulnerability management
OWASP ASVS	2.1.2, 2.4.2	Secrets management & secure coding
PCI DSS 4.0	6.3.2, 6.5.1	No hardcoded credentials in code
SOC 2 (Security)	CC6.1	System configuration and patch control

Key Hardening Checklist

Area	Action	Tool/Method
☐ Secrets Management	Use .env + Secret Manager	GCP Secret Manager / AWS Secrets Manager
☐ Dependency Security	Lock versions & auto-upgrade	Dependabot + Snyk
☐ CI/CD Validation	Block deploy on high vulns	GitHub Actions + Snyk gate
☐ Static Code Analysis	Catch hardcoded secrets	SonarQube + detect-secrets
☐ Runtime Monitoring	Track exploit attempts	Chronicle SIEM / Cloud Logging
☐ Crypto Libraries	Enforce latest OpenSSL	System updates + OS patch
☐ Access Control	Least privilege for API calls	IAM policy reviews
☐ Error Handling	Disable Flask debug mode	app.debug = False in prod

Final Summary

Total Vulnerabilities Identified: 14
Fixable via Upgrades: 9
Manual Code Remediation Required: 3
High-Risk (CVSS ≥ 7): 5
Resolved Automatically via Snyk: None yet applied